

Multilingual AI

Whisper | TTS | AWS Comprehend

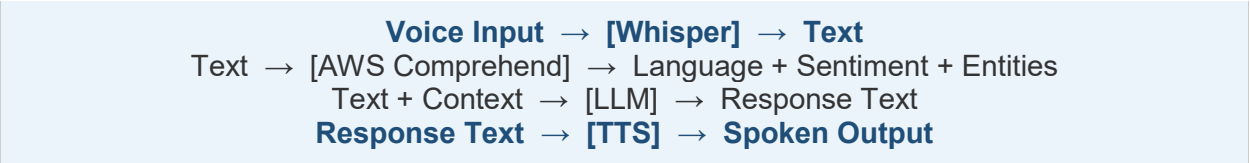
Complete Study Guide
Data Science & AI Engineering

1. The Big Picture — Multilingual AI Pipeline

Multilingual AI enables applications to understand and respond to users in any language, across both text and voice. The full pipeline connects three key tools:

Component	Role in Pipeline
Whisper (OpenAI)	Converts voice audio → text in 99 languages (Speech-to-Text)
LLM (Groq/LLaMA)	Processes text, understands intent, generates response
TTS (gTTS / OpenAI)	Converts response text → spoken audio (Text-to-Speech)
AWS Comprehend	Detects language, sentiment, entities, key phrases from text

Full pipeline flow:



2. Whisper — Speech to Text

Whisper is OpenAI's open-source automatic speech recognition (ASR) model. It converts audio files into text transcripts and supports 99 languages with high accuracy across accents, background noise, and mixed-language speech.

Think of Whisper as a highly trained multilingual stenographer — give it any audio file, get back a clean transcript with language detection.

Whisper Model Sizes

Model	Size	Speed	Best Use Case
tiny	39M params	Fastest	Real-time on CPU, mobile
base	74M params	Fast	Quick demos, prototyping
small	244M params	Medium	Development & testing
medium	769M params	Slow	Production (balanced)
large	1.5B params	Slowest	Max accuracy required

Whisper — Basic Transcription

```
import whisper

# Load model (auto-downloads on first use)
model = whisper.load_model("base")

# Transcribe an audio file
result = model.transcribe("customer_call.mp3")

print(result["text"])          # Full transcript as string
print(result["language"])     # Detected language code e.g. "hi", "es", "fr"

# Force a specific language (faster — skips auto-detection)
result = model.transcribe("audio.mp3", language="es") # Spanish

# Get word-level timestamps
result = model.transcribe("audio.mp3", word_timestamps=True)
for segment in result["segments"]:
    print(f"{segment['start']:.1f}s -> {segment['end']:.1f}s : {segment['text']}")
```

Whisper via Groq API (Faster — Recommended)

Groq hosts Whisper as an API endpoint, giving much faster inference than running locally — no GPU needed.

```
from groq import Groq

client = Groq(api_key='your_groq_api_key')

# Transcribe audio via Groq's Whisper endpoint
with open("customer_call.mp3", "rb") as audio_file:
    transcription = client.audio.transcriptions.create(
        model="whisper-large-v3",
        file=audio_file,
        language="en",          # Optional: omit for auto-detection
        response_format="text"  # "json" for timestamps, "text" for plain string
    )

print(transcription)  # The transcribed text
```

Whisper Key Parameters

Parameter	Description
model	Model size: tiny / base / small / medium / large
language	Force a specific language code (e.g. 'hi', 'es', 'fr') — skip for auto-detect
word_timestamps	True/False — return timing for each word
response_format	text (plain string) or json (includes segments & timestamps)
temperature	0 = deterministic (recommended for transcription)

3. TTS — Text to Speech

Text-to-Speech (TTS) converts AI-generated text responses back into spoken audio. This closes the voice loop: user speaks → AI understands → AI speaks back.

TTS Options Comparison

Tool	Quality	Cost	Best For
gTTS (Google)	Good	Free	Quick prototypes, demos
OpenAI TTS	Excellent	Paid per char	Production voice apps
Coqui TTS	Great	Free / OSS	Self-hosted, custom voices
AWS Polly	Good	Pay-per-char	AWS-native applications

gTTS — Free & Simple

gTTS (Google Text-to-Speech) wraps Google Translate's TTS engine. Easy to use, supports 50+ languages, free for reasonable use.

```
from gtts import gTTS
import os

# English response
text_en = "Hello! How can I help you today?"
tts = gTTS(text=text_en, lang="en", slow=False)
tts.save("response_en.mp3")

# Hindi response
text_hi = "Namaste! Main aapki kaise madad kar sakta hoon?"
tts = gTTS(text=text_hi, lang="hi", slow=False)
tts.save("response_hi.mp3")

# Play on Linux
os.system("mpg321 response_en.mp3")

# Supported language codes: en, hi, es, fr, de, ar, zh, ja, pt, ru ...
```

OpenAI TTS — Best Quality

```
from openai import OpenAI

client = OpenAI(api_key='your_openai_key')

response = client.audio.speech.create(
    model="tts-1",          # tts-1 (fast) or tts-1-hd (higher quality)
    voice="nova",          # alloy, echo, fable, onyx, nova, shimmer
    input="Hello! Your order has been confirmed.",
    speed=1.0              # Range: 0.25 (slow) to 4.0 (fast)
)

response.stream_to_file("output.mp3")
```

OpenAI TTS Voice Options

Voice	Character
alloy	Neutral and balanced — good all-purpose voice
echo	Male, calm and clear
fable	British accent, expressive
onyx	Deep, authoritative male voice
nova	Warm, friendly female voice
shimmer	Soft, gentle female voice

4. AWS Comprehend — Cloud NLP

AWS Comprehend is Amazon's fully managed Natural Language Processing (NLP) service. You send text to it via API — no model to train, host, or maintain. It returns structured insights instantly.

Think of AWS Comprehend as a smart text analyst available on demand — you give it a customer message, it tells you the language, how the customer feels, what they mentioned, and any sensitive information.

AWS Comprehend Features

Feature	What It Returns
Language Detection	Language code + confidence score (e.g. 'hi' at 99.2%)
Sentiment Analysis	POSITIVE / NEGATIVE / NEUTRAL / MIXED + score per category
Entity Recognition	People, places, dates, organizations, quantities found in text
Key Phrase Extraction	The most important phrases from the text
PII Detection	Flags personal info: email, phone number, SSN, credit card, etc.
Syntax Analysis	Part-of-speech tagging (noun, verb, adjective, etc.)

AWS Comprehend Setup & Language Detection

```
import boto3

# Initialize the Comprehend client
comprehend = boto3.client(
    service_name="comprehend",
    region_name="us-east-1",
    aws_access_key_id="YOUR_ACCESS_KEY",
    aws_secret_access_key="YOUR_SECRET_KEY"
)

text = "I am really frustrated with the delayed delivery of my order."

# Detect language
lang_result = comprehend.detect_dominant_language(Text=text)
top_lang = lang_result['Languages'][0]
print(top_lang['LanguageCode'])  # 'en'
print(top_lang['Score'])         # 0.9998 (confidence)
```

Sentiment Analysis

```
# Sentiment analysis
sentiment = comprehend.detect_sentiment(
    Text=text,
    LanguageCode="en"
)

print(sentiment['Sentiment'])          # NEGATIVE
print(sentiment['SentimentScore'])
# {'Positive': 0.01, 'Negative': 0.94, 'Neutral': 0.04, 'Mixed': 0.01}
```

Entity Recognition

```
# Entity recognition
entities = comprehend.detect_entities(
    Text=text,
    LanguageCode="en"
)

for entity in entities['Entities']:
    print(f"{entity['Text']} -> {entity['Type']} (score: {entity['Score']:.2f})")

# Example output:
# 'Amazon' -> ORGANIZATION (score: 0.99)
# 'my order' -> OTHER (score: 0.87)
```

Key Phrase & PII Detection

```
# Key phrase extraction
phrases = comprehend.detect_key_phrases(Text=text, LanguageCode='en')
for phrase in phrases['KeyPhrases']:
    print(phrase['Text'])
# 'delayed delivery', 'my order'

# PII detection (sensitive info)
pii = comprehend.detect_pii_entities(Text=text, LanguageCode='en')
for item in pii['Entities']:
    print(f"{item['Type']}: chars {item['BeginOffset']}-{item['EndOffset']}")
# EMAIL_ADDRESS: chars 45-67
# PHONE: chars 80-92
```

AWS Comprehend Entity Types

Entity Type	Example	Use Case
PERSON	'Gurleen', 'John Smith'	Extract customer/agent names

ORGANIZATION	'Amazon', 'Google'	Brand mentions, competitors
LOCATION	'New Delhi', 'US'	Geographic analysis
DATE	'yesterday', 'Jan 15'	Timeline extraction
QUANTITY	'3 items', '\$50'	Order details, amounts
EVENT	'delayed delivery'	Incident categorization

5. Full Multilingual Voice AI Pipeline

Here is the complete production-ready pipeline combining all three tools with Groq as the LLM provider:

```
from groq import Groq
from gtts import gTTS
import boto3

def multilingual_voice_assistant(audio_file: str) -> dict:
    """
    Full pipeline: Voice -> Whisper -> Comprehend -> LLM -> TTS -> Voice
    """
    groq_client = Groq(api_key='your_groq_key')
    comprehend = boto3.client('comprehend', region_name='us-east-1')

    # STEP 1: Speech -> Text via Groq Whisper
    with open(audio_file, 'rb') as f:
        transcript = groq_client.audio.transcriptions.create(
            model='whisper-large-v3', file=f
        )
    user_text = transcript.text
    print(f'Transcribed: {user_text}')

    # STEP 2: Detect language & sentiment via AWS Comprehend
    lang_result = comprehend.detect_dominant_language(Text=user_text)
    detected_lang = lang_result['Languages'][0]['LanguageCode']
    sentiment = comprehend.detect_sentiment(
        Text=user_text, LanguageCode=detected_lang
    )
    print(f'Language: {detected_lang}, Sentiment: {sentiment["Sentiment"]}')

    # STEP 3: Generate response via Groq LLM
    prompt = f"""
    The user said (in language {detected_lang}): {user_text}
    Their sentiment is: {sentiment['Sentiment']}
    Please reply helpfully in the same language ({detected_lang}).
    """
    response = groq_client.chat.completions.create(
        model='llama3-70b-8192',
        messages=[{'role': 'user', 'content': prompt}]
    )
    reply_text = response.choices[0].message.content

    # STEP 4: Text -> Speech via gTTS
    tts = gTTS(text=reply_text, lang=detected_lang)
    tts.save('response.mp3')

    return {
        'transcript': user_text,
        'language': detected_lang,
```

```
'sentiment': sentiment['Sentiment'],  
'reply': reply_text,  
'audio_file': 'response.mp3'  
}
```

6. Q&A Study Cards

Q: What is Whisper and what makes it different from other speech-to-text tools?

A: Whisper is OpenAI's open-source ASR (Automatic Speech Recognition) model that converts audio to text in 99 languages. What makes it different: it handles accents, background noise, and mixed-language speech very well. It also auto-detects the language without you specifying it. It's available both locally (install the library) and via Groq's API for much faster inference.

Q: What are the 5 Whisper model sizes and when would you choose each?

A: tiny (fastest, basic accuracy — real-time on CPU/mobile), base (fast, good — quick demos), small (medium speed, better — development), medium (slow, great — production balance), large (slowest, best — when accuracy is critical). Rule of thumb: start with 'base' for development, use 'medium' or 'large' in production.

Q: What is the difference between STT (Speech-to-Text) and TTS (Text-to-Speech)?

A: STT (Speech-to-Text) converts audio/voice input into text — this is what Whisper does. TTS (Text-to-Speech) converts text back into spoken audio — this is what gTTS, OpenAI TTS, or AWS Polly do. Together they form the full voice loop: user speaks → STT → AI processes → TTS → AI speaks back.

Q: What are the 5 main features of AWS Comprehend?

A: 1) Language Detection — identifies what language text is written in with a confidence score. 2) Sentiment Analysis — classifies text as POSITIVE, NEGATIVE, NEUTRAL, or MIXED. 3) Entity Recognition — extracts people, places, organizations, dates, quantities. 4) Key Phrase Extraction — identifies the most important phrases. 5) PII Detection — flags personally identifiable information like emails, phone numbers, SSNs.

Q: What is the difference between Sentiment Analysis and Entity Recognition in AWS Comprehend?

A: Sentiment Analysis tells you HOW the customer feels (POSITIVE/NEGATIVE/NEUTRAL/MIXED) — it's about emotion and tone. Entity Recognition tells you WHAT was mentioned in the text — specific named things like 'Amazon' (ORGANIZATION), 'John' (PERSON), 'yesterday' (DATE). They answer different questions: feeling vs. content.

Q: Why would you use Groq's Whisper API instead of running Whisper locally?

A: Speed and infrastructure. Running Whisper locally requires a GPU for decent performance. Groq's hardware (LPU — Language Processing Unit) is purpose-built for inference and can transcribe audio much faster than a typical developer machine. It also removes the need to manage GPU infrastructure. The downside: API costs and sending audio data to a third-party service.

Q: In the multilingual pipeline, why do we run AWS Comprehend BEFORE sending text to the LLM?

A: We need to know the detected language before asking the LLM to respond — so the LLM knows which language to reply in. We also pass the sentiment to the LLM so it can adjust its tone (empathetic for NEGATIVE, enthusiastic for POSITIVE). This structured metadata makes the LLM's response more accurate and contextually appropriate.

Q: What is PII detection and why is it critical in production AI systems?

A: PII (Personally Identifiable Information) detection identifies sensitive personal data in text — email addresses, phone numbers, SSNs, credit card numbers, etc. In production, you need to detect and redact PII before: logging conversations (GDPR compliance), sending data to third-party LLM APIs, storing in databases. AWS Comprehend's PII detection flags the type and character position of each PII item found.

Q: What is the gTTS library and how does it work?

A: gTTS (Google Text-to-Speech) is a Python library that wraps Google Translate's TTS engine. You pass text and a language code, it returns an MP3 audio file. It's free for reasonable use, supports 50+ languages, and is the simplest way to add voice output to a prototype. Limitation: it requires an internet connection (calls Google's API) and the voice quality is robotic compared to OpenAI TTS.

Q: How would you build a multilingual customer support bot using these 3 tools?

A: 1) User sends a voice message → Whisper (via Groq) transcribes it to text. 2) AWS Comprehend detects language + sentiment + entities (e.g., product names, dates mentioned). 3) Pass transcript + detected language + sentiment context to Groq LLM → generate reply in the same language. 4) Convert reply to speech using gTTS or OpenAI TTS in the detected language. 5) Return audio response to user. The entire pipeline auto-adapts to any language the user speaks.

7. Quick Reference Cheat Sheet

Whisper Quick Reference

Operation	Code
Load model	<code>model = whisper.load_model('base')</code>
Transcribe file	<code>result = model.transcribe('audio.mp3')</code>
Get text	<code>result['text']</code>
Get language	<code>result['language']</code>
Force language	<code>model.transcribe('audio.mp3', language='hi')</code>
Via Groq API	<code>client.audio.transcriptions.create(model='whisper-large-v3', file=f)</code>

gTTS Quick Reference

Operation	Code
Create speech	<code>tts = gTTS(text='Hello', lang='en')</code>
Save to file	<code>tts.save('output.mp3')</code>
Hindi TTS	<code>gTTS(text='Namaste', lang='hi')</code>
Spanish TTS	<code>gTTS(text='Hola', lang='es')</code>
Slow speech	<code>gTTS(text='Hello', lang='en', slow=True)</code>

AWS Comprehend Quick Reference

Operation	Code
Detect language	<code>comprehend.detect_dominant_language(Text=text)</code>
Sentiment analysis	<code>comprehend.detect_sentiment(Text=text, LanguageCode='en')</code>
Entity recognition	<code>comprehend.detect_entities(Text=text, LanguageCode='en')</code>
Key phrases	<code>comprehend.detect_key_phrases(Text=text, LanguageCode='en')</code>

PII detection

```
comprehend.detect_pii_entities(Text=text,  
LanguageCode='en')
```

Language Code Reference

Language	Code	Supported By
English	en	Whisper, gTTS, AWS Comprehend
Hindi	hi	Whisper, gTTS, AWS Comprehend
Spanish	es	Whisper, gTTS, AWS Comprehend
French	fr	Whisper, gTTS, AWS Comprehend
Arabic	ar	Whisper, gTTS, AWS Comprehend
Chinese	zh	Whisper, gTTS (zh-cn/zh-tw)
German	de	Whisper, gTTS, AWS Comprehend
Japanese	ja	Whisper, gTTS, AWS Comprehend